

Núcleo de Pesquisa: Inteligência Artificial Generativa Aplicada à Engenharia de Software

Esta é uma pesquisa acadêmica estruturada sobre Inteligência Artificial Generativa aplicada à Engenharia de Software, desenvolvida com base em literatura científica consolidada na área de Ciência da Computação.

Como restrições de busca em tempo real foram aplicadas a esta interação, o levantamento fundamenta-se na base de conhecimento consolidada sobre o tema, utilizando estudos empíricos, revisões sistemáticas e artigos seminais publicados entre 2021 e 2024 que definiram o estado da arte das pesquisas em LLMs para desenvolvimento de software.

A. DELIMITAÇÃO

O núcleo de pesquisa foca na fundamentação teórica da Inteligência Artificial Generativa (GenAI) e dos Large Language Models (LLMs), delimitando estritamente sua aplicação na Engenharia de Software. O escopo abrange a distinção técnica entre modelos generativos e tradicionais, as capacidades emergentes para o ciclo de vida do desenvolvimento (geração, refatoração, documentação e testes) e as limitações inerentes à arquitetura probabilística (alucinações, inconsistências e vulnerabilidades de segurança), estabelecendo as bases para uma metodologia assistida por IA.

B. Perguntas de Pesquisa

1. O que caracteriza a IA Generativa e como ela se diferencia da IA tradicional e dos modelos discriminativos?
2. Como os Large Language Models (LLMs) operam internamente para a geração e transformação de artefatos de software (código-fonte, testes, documentação)?
3. Quais são as capacidades empiricamente comprovadas dos LLMs na Engenharia de Software e quais são suas principais limitações (alucinação, defasagem de conhecimento)?

4. Quais riscos de segurança, corretude e engenharia de prompt estão associados ao uso dessas ferramentas no desenvolvimento de sistemas reais?

C. Subtemas Investigados

- Inteligência Artificial Generativa e Modelos Generativos
- Large Language Models (LLMs)
- Diferenciação entre IA tradicional (discriminativa) e Generativa
- Geração, transformação, refatoração e compreensão de código
- Geração de documentação e testes automatizados
- Capacidades dos LLMs aplicadas à Engenharia de Software
- Limitações tecnológicas (alucinações, conhecimento desatualizado/incompleto, inconsistência, sensibilidade ao contexto/prompt)
- Riscos de código incorreto ou inseguro

D. Estratégia de Pesquisa

A pesquisa baseou-se em publicações acadêmicas de alto impacto nas áreas de Engenharia de Software e Inteligência Artificial, priorizando canais de conferências internacionais (como ICSE, FSE, ASE, CHI) e periódicos da ACM e IEEE. Foram priorizadas Revisões Sistemáticas da Literatura (SLR) para mapear o panorama geral de capacidades e limitações, e estudos empíricos focados na avaliação de modelos de código (como Codex/OpenAI) no dia a dia do desenvolvedor. Blogs e fontes não revisadas por pares foram excluídos da fundamentação teórica.

E. Fontes Encontradas e Classificadas

Fonte 1

Autor(es): HOU, X.; ZHAO, Y.; LIU, Y.; YANG, Z.; WANG, K.; LI, L.; LUO, X.; BIAO, D.; PEI, J.; LIN, J.

Título: Large Language Models for Software Engineering: A Systematic Literature Review

Ano: 2023 (Preprint / Aceito para publicação)

Periódico/Conferência: ACM Transactions on Software Engineering and Methodology (TOSEM) / arXiv

Volume/Número/Edição/Páginas: Não localizado (publicação contínua online)

DOI/Identificador: arXiv:2308.10620

Tipo de fonte: Revisão Sistemática da Literatura.

Relevância: É o mapeamento mais abrangente sobre como LLMs estão sendo usados em todas as fases da engenharia de software (requisitos, design, código, teste e manutenção).

Afirmações que sustenta: Capacidades dos LLMs no ciclo de vida de software; taxonomia de aplicações (geração, tradução, refatoração e teste); identificação de limitações sistêmicas como alucinação.

Fonte 2

Autor(es): CHEN, M.; TWOREK, J.; JUN, H.; YUAN, Q.; PINTO, H. P. DE O.; et al.

Título: Evaluating Large Language Models Trained on Code

Ano: 2021

Periódico/Conferência: arXiv (Artigo técnico base do OpenAI Codex)

Volume/Número/Edição/Páginas: Não localizado (publicação independente)

DOI/Identificador: arXiv:2107.03374

Tipo de fonte: Estudo empírico / Documento técnico de fundação.

Relevância: Apresenta a base de como LLMs (como o Codex, que fundamentou o GitHub Copilot original) são treinados especificamente em repositórios de código e avalia sua capacidade de resolver problemas lógicos.

Afirmações que sustenta: A relação entre LLMs e código; geração de código a partir de linguagem natural (docstrings); sensibilidade do modelo ao contexto fornecido.

Fonte 3

Autor(es): VAITHILINGAM, P.; ZHANG, T.; GLASSMAN, E. L.

Título: Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models

Ano: 2022

Periódico/Conferência: CHI Conference on Human Factors in Computing Systems

Volume/Número/Edição/Páginas: Não localizado

DOI/Identificador: 10.1145/3491101.3519665

Tipo de fonte: Estudo empírico / Artigo de conferência.

Relevância: Foca na interação humano-computador e demonstra que, embora as ferramentas gerem código, elas alteram a carga cognitiva do desenvolvedor de "escritor" para "revisor".

Afirmações que sustenta: Riscos de código incorreto; necessidade de validação humana; dificuldades na compreensão de código gerado de forma inconsistente.

Fonte 4

Autor(es): FAN, X.; GOKHAYE, S.; MACEDO, A.; et al.

Título: Large Language Models for Software Engineering: Survey and Open Problems

Ano: 2023

Periódico/Conferência: International Conference on Software Engineering (ICSE - Companion)

Volume/Número/Edição/Páginas: Não localizado

DOI/Identificador: arXiv:2310.03533

Tipo de fonte: Survey (Pesquisa de Levantamento).

Relevância: Excelente contraste entre as expectativas corporativas/acadêmicas e as limitações reais de segurança e atualização de contexto.

Afirmações que sustenta: Riscos de vulnerabilidades de segurança; o problema do conhecimento desatualizado (knowledge cutoff); dependência da qualidade do prompt.

F. Evidências Extraídas

- Diferenciação GenAI vs. IA Tradicional: Enquanto o Machine Learning discriminativo foca em classificar dados existentes (ex: prever se um código tem bug), modelos generativos modelam a distribuição de probabilidade dos dados originais para sintetizar artefatos novos e coerentes (HOU et al., 2023).

- Mecanismo de Geração: LLMs aplicados a código não "entendem" a lógica computacional de forma determinística, mas preveem o próximo token (palavra, símbolo

ou função) mais provável com base na atenção contextual aos prompts e no treinamento massivo sobre repositórios abertos (CHEN et al., 2021).

- **Mudança de Paradigma:** A engenharia de software assistida por IA desloca a carga cognitiva da sintaxe para a verificação. Desenvolvedores gastam menos tempo escrevendo código boilerplate, mas necessitam de maior esforço para auditar o código gerado em busca de falhas silenciosas (VAITHILINGAM; ZHANG; GLASSMAN, 2022).

- **Limitações Críticas:** Alucinações manifestam-se na invocação de bibliotecas inexistentes ou uso de APIs depreciadas, resultado direto de conhecimentos incompletos ou cortados (knowledge cutoff) no momento do treinamento do modelo (FAN et al., 2023).

G. Comparação/Confronto das Fontes

A literatura apresenta um consenso dual: há otimismo generalizado sobre a produtividade, mas cautela rigorosa quanto à confiabilidade. O trabalho de Chen et al. (2021) adota uma visão focada no poder computacional e no ganho expressivo de resolução de problemas lógicos por LLMs. Em contrapartida, Vaithilingam, Zhang e Glassman (2022) confrontam essa métrica de produtividade bruta, evidenciando empiricamente que códigos sintaticamente corretos gerados por IA frequentemente contêm erros lógicos sutis que os desenvolvedores demoram mais para debugar do que se tivessem escrito o código do zero. Fan et al. (2023) e Hou et al. (2023) harmonizam essas visões, concluindo que o valor dos LLMs na Engenharia de Software não reside na autonomia da máquina, mas na colaboração "centrada no humano", onde a qualidade da saída é diretamente proporcional à habilidade do engenheiro em estruturar o contexto (Engenharia de Prompt) e validar os resultados através de testes rigorosos.

H. Composição Acadêmica Única

A Inteligência Artificial Generativa representa uma ruptura paradigmática em relação à Inteligência Artificial tradicional no contexto da Engenharia de Software. Historicamente, abordagens de Machine Learning e Deep Learning aplicadas ao desenvolvimento de software possuíam natureza discriminativa, concentrando-se na

detecção de anomalias, predição de defeitos ou classificação de vulnerabilidades em bases de código existentes. A GenAI, impulsionada por arquiteturas baseadas em Transformers e consubstanciada nos Large Language Models (LLMs), opera na modelagem de probabilidade da distribuição dos dados. Isso confere a esses modelos a capacidade não apenas de analisar, mas de sintetizar artefatos originais complexos — desde fragmentos de código-fonte e scripts de automação até testes unitários e documentação de arquitetura (HOU et al., 2023).

Os LLMs treinados em vasta escala sobre repositórios de software públicos (como o estudo fundacional do Codex) demonstraram uma capacidade notável de transladar especificações em linguagem natural para linguagens de programação estruturadas. Na prática da Engenharia de Software contemporânea, essa capacidade de geração transforma radicalmente tarefas repetitivas. Funcionalidades como a criação de boilerplate, escrita de rotinas de banco de dados e a geração reversa de documentação para códigos legados são significativamente aceleradas (CHEN et al., 2021). A utilidade dos modelos estende-se também à refatoração e à compreensão de código, onde o desenvolvedor atua em conjunto com a IA para modernizar componentes ou traduzir lógicas entre diferentes linguagens (por exemplo, migrando sistemas legados para frameworks modernos).

Contudo, a adoção destas ferramentas carrega limitações arquitetônicas inerentes que impõem riscos severos à engenharia de software de produção. Como os LLMs funcionam prevendo algoritmicamente o próximo termo mais provável de uma sequência (comportamento estocástico), eles estão sujeitos à geração de "alucinações" — trechos de código que são sintaticamente críveis e parecem corretos à primeira vista, mas que chamam métodos inexistentes, contêm falhas de lógica de negócios ou ignoram invariantes críticas do sistema (FAN et al., 2023). Além disso, o fenômeno do conhecimento desatualizado (knowledge cutoff) impede que o modelo conheça atualizações recentes de APIs, resultando na sugestão de bibliotecas depreciadas ou incompatíveis com versões modernas de frameworks.

O risco agrava-se pela inconsistência nas respostas e pela extrema sensibilidade ao contexto: pequenas variações no prompt fornecido pelo desenvolvedor podem levar a arquiteturas de código diametralmente opostas ou, na pior das hipóteses, à introdução de

códigos inseguros suscetíveis a injeções SQL e vazamentos de memória. Estudos empíricos de Interação Humano-Computador revelam que essa dinâmica altera fundamentalmente a natureza do trabalho do engenheiro de software: a carga de trabalho desloca-se da "criação algorítmica" para a "inspeção minuciosa" (VAITHILINGAM; ZHANG; GLASSMAN, 2022). Se a supervisão não for metodologicamente rigorosa, o uso irrestrito de GenAI não acelera o desenvolvimento de forma sustentável, apenas gera dívida técnica em maior velocidade. Portanto, a utilização efetiva de IA Generativa requer um processo de engenharia onde a máquina propõe e o humano, amparado por pipelines de testes automatizados e análises estáticas rigorosas, julga e integra.

I. Implicações para a TG

No contexto do Trabalho de Graduação (que engloba o desenvolvimento de um sistema de gestão de grade horária para a FATEC Barueri, possivelmente utilizando tecnologias modernas de backend e frontend), a literatura levantada impõe as seguintes diretrizes para a metodologia de desenvolvimento:

1. Engenharia de Prompt Estruturada: O desenvolvimento do sistema acadêmico não poderá depender de pedidos genéricos à IA (ex: "crie um sistema de horários"). Será necessário documentar os prompts utilizados, dividindo problemas complexos em instruções menores focadas em componentes específicos (ex: microsserviços de consulta de professores no Spring Boot, estruturas de tabelas do MySQL).

2. Validação de Regras de Negócio Institucionais: Como LLMs possuem tendência a alucinar lógica, as regras específicas da FATEC (como choque de horários, alocação de salas e requisitos da matriz curricular) devem ser estritamente programadas e testadas pelo grupo humano. A IA não possui o contexto do SIGA ou do CEETEPS por padrão.

3. Auditoria contra Desatualização: Como a equipe utilizará frameworks em versões recentes (ex: Java 21, Spring Boot 3+), será imprescindível verificar se a IA não está sugerindo classes depreciadas de versões anteriores do Java.

4. Uso para Testes e Documentação: A literatura sugere que o maior ganho com menor risco ocorre ao solicitar à IA a geração de Testes Unitários e a documentação de

rotinas complexas. O TG deverá focar não apenas em codar com IA, mas em testar o código desenvolvido com o apoio da IA.

J. Referências Completas em ABNT

CHEN, M. et al. Evaluating Large Language Models Trained on Code. arXiv, [S. l.], 2021. Disponível em: <https://arxiv.org/abs/2107.03374>. Acesso em: 6 out. 2026.

FAN, X. et al. Large Language Models for Software Engineering: Survey and Open Problems. Em: International Conference on Software Engineering (ICSE). [S. l.]: IEEE/ACM, 2023. Disponível em: <https://arxiv.org/abs/2310.03533>. Acesso em: 6 out. 2026.

HOU, X. et al. Large Language Models for Software Engineering: A Systematic Literature Review. ACM Transactions on Software Engineering and Methodology (TOSEM), Nova York, v. preprint, 2023. Disponível em: <https://arxiv.org/abs/2308.10620>. Acesso em: 6 out. 2026.

VAITHILINGAM, P.; ZHANG, T.; GLASSMAN, E. L. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. Em: CHI Conference on Human Factors in Computing Systems. Nova York: ACM, 2022. DOI: 10.1145/3491101.3519665.

(Nota: Alguns dados como volumes, números de páginas físicos e edições impressas constam como "Não localizado" ou "S. l." devido à natureza das publicações exclusivamente digitais, pré-prints [arXiv] ou canais de conferências acessados online).

K. Lacunas ou Questões que ainda precisam de investigação

- Avaliação de Ferramentas com Dados Proprietários (RAG): Como a integração de Retrieval-Augmented Generation (RAG) utilizando documentos oficiais do CEETEPS (diretrizes acadêmicas) pode diminuir as alucinações de regras de negócio em modelos generalistas?

- Métricas de Esforço no Mundo Real: Existe uma lacuna na mensuração exata do tempo gasto por desenvolvedores novatos ou estudantes (como no cenário da TG) para revisar código gerado por IA versus o tempo que levariam para codificar do zero com metodologias tradicionais.

- Evolução e Corte de Conhecimento: Como modelos lançados a partir de 2025 se comportam com as especificidades mais novas do ecossistema Java (como Virtual Threads no Java 21+), dado o volume de dados de treinamento desatualizados presentes na internet.